

Адаптивный аккомпанемент в реальном времени:

от классических трекеров партитуры
к RL-агенту опережающего темпа

Гибридная архитектура для виртуального оркестра

Никита Новицкий | Никита Борисов

Центральный Университет (Т-Банк), мастерская MusicTech

Мультидисциплинарная молодёжная конференция

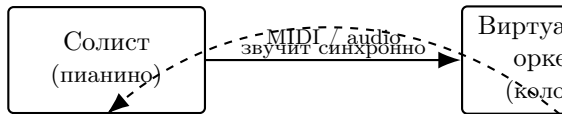
«Научный Телеграф», 2026

Проблема: аккомпаниатор, а не фонограмма

- Солисту-пианисту перед концертом нужно репетировать с оркестром.
- Живой оркестр на каждую репетицию — недоступно (организация, бюджет, время).
- Фиксированная фонограмма не работает: она не подстраивается под темп, артикуляцию и rubato солиста.
- Цель: виртуальный оркестр, который играет заданное сопровождение и подстраивается под солиста в реальном времени.

играет в своём темпе

подстра

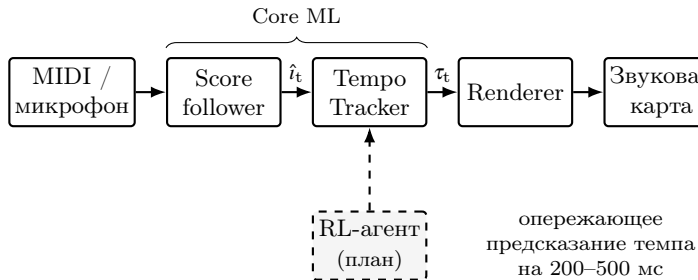


Не фонограмма — аккомпаниатор.

Что технически делает система

Задача формализуется как score-following: по поступающему MIDI/аудио-потoku и заранее известной партитуре в реальном времени определять текущую позицию исполнителя.

- Вход: партитура (известна) + поток нот солиста (realtime).
- Выход: индекс ноты $\hat{i}_t$ + темповый коэффициент τ_t .
- Бюджет задержки: ≤ 20 мс (порог восприятия рассинхронизации, Cont 2010).



Core ML (HMM/HSMM/OLTW) и TempoTracker — классическое ядро. RL-агент дополняет ядро опережающим предсказанием темпа на 200–500 мс.

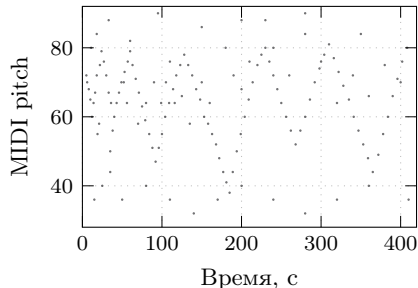
Данные: формат партитуры

Партитура хранится в JSON-формате `score.json`. Одна запись = одно скрытое состояние HMM/HSMM (может быть аккордом).

```
{  
  "index": 0,  
  "pitches": [60, 64, 67],  
  "nominal_onset": 2.999,  
  "nominal_duration": 1.499  
}
```

- `generated_dataset/` — 4 синтетические пары;
- `midi/library/rach_solo/` — Рахманинов, 3791 state;
- для RL — импортер ASAP (Foscarin 2020): 222 пьесы, 1067 исполнений с note-level разметкой.

Рахманинов: 3791 state партитуры



Рахманинов, 1-я часть концерта №2. Каждая точка — состояние партитуры (верхняя нота аккорда).

Live-поток от солиста — последовательность MIDI-событий:

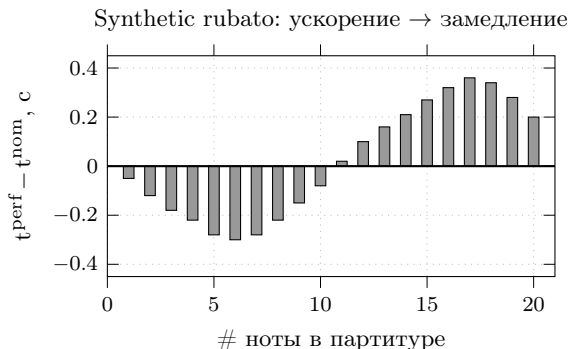
```
{"pitch": 64, "timestamp": 1.234, "velocity": 88}
```

```
{"pitch": 67, "timestamp": 1.481, "velocity": 95}
```

...

В реальном исполнении timestamps не совпадают с nominal_onset из-за rubato, ускорений, замедлений и ошибок.

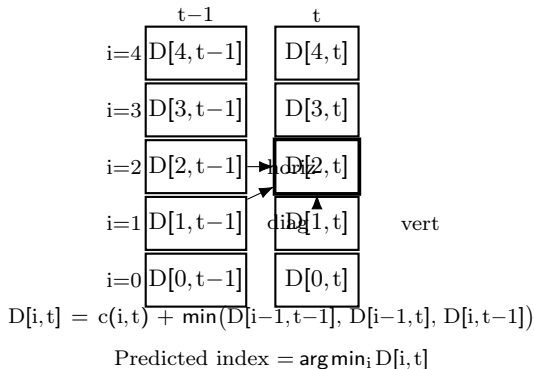
Это и есть challenge задачи: трекер должен оставаться синхронным, даже когда времени между нотами на лету меняется в 2–3 раза.



Типичный rubato: пианист сначала ускоряется (отрицательные дельты), потом замедляется

On-Line Time Warping — инкрементальный вариант DTW.

- Локальная стоимость: $c(i, t) = |p_i - o_t|$.
- Рекурсия:
$$D[i, t] = c(i, t) + \min(D[i-1, t-1], D[i-1, t], D[i, t-1]).$$
- Предсказание: $\hat{i}_t = \arg \min_i D[i, t]$.
- Память $O(N)$ — хранятся только две колонки.
- Монотонизация + RunCount fail-safe: если позиция k событий не двигается, форсируем шаг вперёд.



Predicted index = argmin по текущему столбцу.

Плюсы: простота, устойчивость к rubato, всегда выдаёт ответ. Минусы: нет confidence, плохо ловит структурные ошибки.

Скрытая Марковская модель по партитуре.

- Состояния: $i \in \{0, \dots, N-1\}$ — индексы нот.
- Эмиссия (гауссиана по pitch в полутонах):

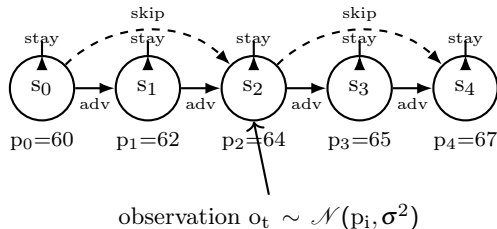
$$b_i(o) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{1}{2}\left(\frac{o-p_i}{\sigma}\right)^2\right)$$

- Переходы $a_{j,i} \in \{\text{stay}, \text{advance}, \text{skip}\}$, веса зависят от elapsed в state.
- Forward-обновление:

$$\alpha_{t+1}(i) = b_i(o_{t+1}) \sum_j \alpha_t(j) a_{j,i}$$

- $\hat{i}_t = \arg \max_i \alpha_t(i)$, confidence = $\max_i \alpha_t(i)$.

Что добавляет к OLTW: распределение по позициям $\Rightarrow$ есть confidence для fusion с другими моделями.



Цепочка состояний. Stay-петли, advance-шаги, skip-через-одно. Эмиссия — гауссиана по pitch.

Baseline 3: HSMM (Cont 2010, эвристика)

Hidden Semi-Markov Model — расширение HMM с явной моделью длительности.

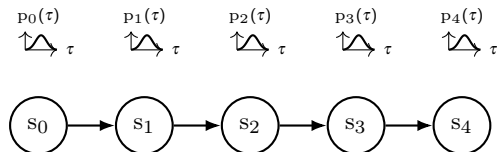
Канонический вид (Cont 2010):

$$\alpha_t(i) = \sum_{\tau=1}^D \sum_j \alpha_{t-\tau}(j) a_{j,i} p_i(\tau) \prod_{k=t-\tau+1}^t b_i(o_k)$$

где $p_i(\tau)$ — распределение длительности state i .

В нашем коде (`hsmm_follower.py`) реализована эвристика: переходы пересчитываются от $r = \text{elapsed} / \text{nominal_duration}$. Четыре исхода:

stay / advance / skip / leap
пропуск 2 состояний



HMM + распределение длительности $p_i(\tau)$
 $\Rightarrow$ переходы зависят от elapsed в state

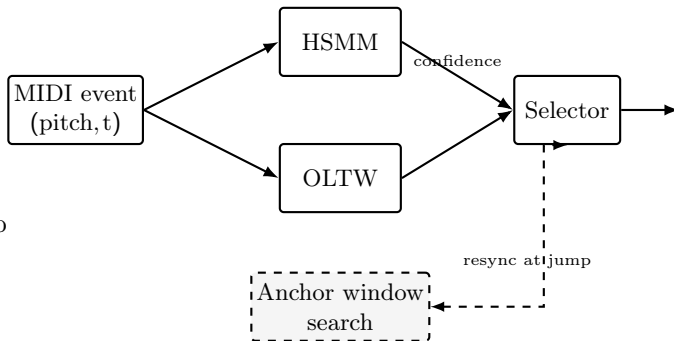
HMM + распределение длительности $p_i(\tau)$
над каждым state. Переходы зависят от elapsed в state.

Добавляет к HMM: учёт длительностей $\Rightarrow$ правильное поведение при rubato и лишних нотах.
Ограничение: наш HSMM — эвристика, не канонический Cont. Плановое улучшение.

Baseline 4: Hybrid Fusion

HybridScoreFollower — параллельно работают HSMM и OLTW плюс якорный resync.

1. HSMM: вероятностный ответ с confidence.
2. OLTW: жёсткий fallback-ответ.
3. Anchor search: последние K событий ищутся как окно в score через template-matching по pitch + temp-fitting.



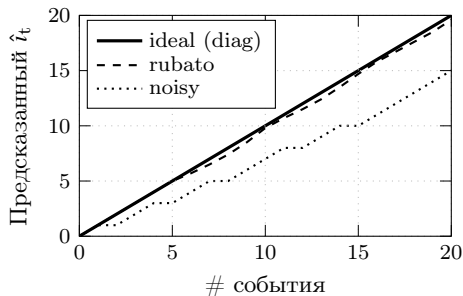
Правило выбора:

- $\text{conf} > \theta \Rightarrow \text{HMM}$;
- иначе $\Rightarrow \text{OLTW}$;
- якорь нашёл сильное совпадение $\Rightarrow \text{resync}$ обоих.

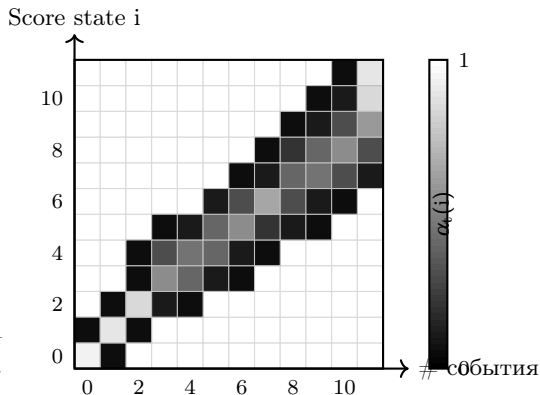
Это «классический трекер» в смысле тезисов конференции — поверх него ставится RL-слой.

Результаты baseline на синтетике

Прогон трёх трекеров на синтетических сценариях: ideal (гамма ровно по нотам), rubato (тот же pitch, expressive timing), noisy (лишние ноты, опечатки на полутон, пропуски).



OLTW: на ideal/rubato — диагональ. На noisy — ступеньки от форсированных шагов RunCount.



НММ α -heatmap на noisy: уверенность «размывается» по соседним состояниям при ошибках.

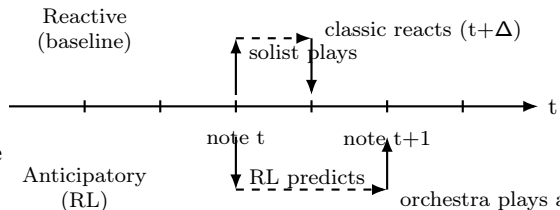
Почему baseline недостаточно

Три фундаментальных ограничения классических трекеров.

1. Реактивность. TempoTracker считает темп постфактум — по уже сыгранным нотам. Пока оркестр «узнаёт», что солист ускорился, тот уже играет следующую фразу. Нужно опережающее предсказание темпа.

2. Жёсткие правила. Все эвристики (пороги confidence, deadzone темпа, anchor margin) подобраны вручную и плохо переносятся между исполнителями.

3. Не используют контекст. Trackers видят одну ноту за раз; они не знают, как менялась confidence, какая была динамика темпа за последние 500 мс, где находится текущая фраза в партитуре.

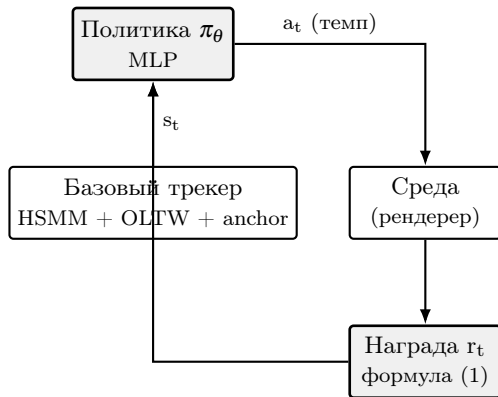


Reactive vs anticipatory tempo control.
Класс. трекер реагирует с задержкой Δ ;
RL-агент предсказывает темп на 200–500
мс вперёд.

Решение тезисов: лёгкий RL-агент, который видит весь вектор состояния трекера и 11/18

Предлагаемая архитектура (РИС. 1 тезисов)

Идея: не заменять классический трекер, а подвесить рядом лёгкий RL-агент. Базовый трекер остаётся ядром. Агент читает состояние трекера и историю темпа, выдаёт темповый коэффициент $a_t \in [0.5, 2.0]$ на горизонте 200–500 мс.



Новая ниша:

- Dorfer 2018 — RL заменяет трекер целиком.
- Peter 2023 — RL для symbolic alignment.
- RL-Duet, ReaLchords — RL генерирует ноты.
- Наш подход — RL поверх работающего трекера, только темп, без замены классики.

Если RL-агент молчит — система

$a_t \in [0.5, 2.0]$ — единственное число на выходе агента, деградирует до проверенного гибрида, а не ломается.

Состояние $s_t \in \mathbb{R}^{18}$ (не зависит от длины пьесы):

$$s_t = (\hat{\alpha}_t, \tau_{t-K:t}, e_{t-K:t}, \hat{\phi}(t)/N)$$

где $\hat{\alpha}_t$ — 7-числовая сжатка HSMM-распределения (max, entropy, argmax/N, top-3 mass, 3 относительных индекса), $\tau_{t-K:t}$ — последние K оценок темпа, $e_{t-K:t}$ — эмиссионные ошибки, $\hat{\phi}(t)/N$ — относительная позиция.

Действие: $a_t \in [0.5, 2.0]$ — темповый коэффициент.

Награда (формула (1) тезисов):

$$r_t = \underbrace{-|t_t^{\text{render}} - t_t^{\text{perf}}|}_{\text{рассинхр.}} \underbrace{- \lambda L_{\text{align}}(t)}_{\text{ошибка трекера}} \underbrace{- \mu (a_t - a_{t-1})^2}_{\text{резкий перепад}}$$

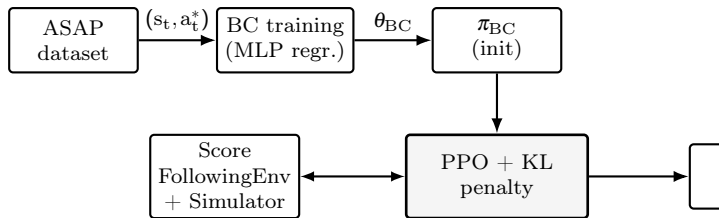


Обучение: двустадийная схема $BC \rightarrow PPO$

PPO с нуля плохо стартует на сложной непрерывной задаче. Используем стандартную схему $BC \rightarrow PPO + KL$ (Sequence Tutor, Jaques 2017).

Стадия А — Behavior Cloning на ASAP:

1. Считаем оракульный темп $a_t^* = \frac{\Delta\Phi_{\text{nominal}}}{\Delta\Phi_{\text{real}}}$ на скользящем окне.
2. Регрессия $s_t \mapsto a_t^*$ — MLP 2×64 .
3. Получаем стартовую политику π_{BC} .



ASAP даёт инициализацию, PPO дотачивает в симуляторе.

Стадия В — PPO + KL:

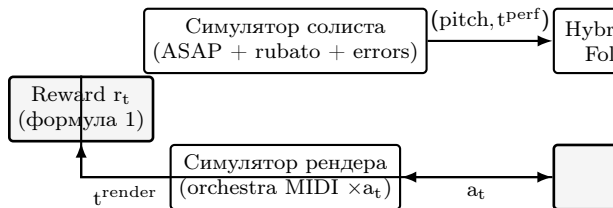
1. Среда ScoreFollowingEnv (Gymnasium) поверх живого HybridScoreFollower.
2. Симулятор солиста (rubato по Repp 1995).
3. Награда r_t из формулы (1).

Симулятор солиста и рендера

Для PPO нужны эпизоды — (state, action, reward) в больших количествах. На реальных ASAP-записях их мало (~ 1000 исполнений). Решение — симулятор.

Симулятор солиста:

- база — реальное ASAP-исполнение или партитура;
- параметрическое rubato (Repp 1995): локальное растяжение времени по фразам;
- сэмплирование ошибок (вставка/замена/пропуск по Nakamura 2015);
- диверсивность $\Rightarrow$ агент не переобучается на конкретных записях.



Замкнутый цикл: симулятор $\rightarrow$ трекер $\rightarrow$ политика $\rightarrow$ рендер $\rightarrow$ reward $\rightarrow$ обратно в политику.

Симулятор рендера:

- детерминированное проигрывание orchestra MIDI с темпом a_t ;
- t_t^{render} берётся напрямую из времени события после temp-rescaling.

Что готово и план до защиты

Готово (наша работа поверх baseline):

- Архитектурная карта репозитория, слои в пакете musictech/ (13 подпакетов).
- DTO под RL: AlphaSummary, RLObservation, RLAction, RLReward.
- 9 ноутбуков с пошаговым разбором каждого компонента — от формата партитуры до сборки s_t из живого HybridScoreFollower.
- Проверено: s_t собирается из текущего стека за один вызов, размерность 18, не зависит от длины пьесы.

План до защиты (приоритет):

1. Импортёр ASAP в наш формат (musictech/datasets/asap.py) — блокёр для всего обучения.
2. Вынос метрик в musictech/evaluation/ (alignment accuracy, onset-error, recovery latency).
3. Среда Gymnasium + симулятор солиста.
4. BC на ASAP, потом PPO + KL.

Baseline
готов

Архитектура
готова

ASAP-
импортёр
+ метрики

BC +
симулятор

PPO +
интеграция
в живой
стек

Выводы и дальнейшая работа

Сделано:

- Систематизировали baseline (OLTW, HMM, HSMM, Hybrid) — все три классических подхода работают на синтетических сценариях.
- Реструктурировали кодовую базу под расширения, заложили типизированные DTO под RL.
- Воспроизводимые ноутбуки-демонстрации каждого компонента.

Ключевая идея:

RL-агент не заменяет классический трекер, а дополняет его как лёгкий слой опережающего предсказания темпа.

Это новая ниша, в литературе не покрытая.

Дальше:

- Подключить ASAP, обучить BC $\rightarrow$ PPO по описанной схеме.
- Сравнить с baseline по RMS render-perf delay и tempo jerk.
- Адаптация под конкретного исполнителя (fine-tune политики после нескольких сессий) — главное прикладное преимущество RL.

Ограничения:

- Бюджет 20 мс end-to-end — нужно подтвердить на крупных пьесах с beam-HSMM.

Спасибо за внимание

Вопросы?

Репозиторий: github.com/Nizier193/music-tech

Статья: [article/main.pdf](#)

Тезисы: [article/тезисы.pdf](#)

Код: [src/musictech-app/](#)

Никита Новицкий | Никита Борисов
Центральный Университет, мастерская MusicTech